



Software Engineering

Sprint 0 Report

CodingFactory Platform

Submitted by:

Malek BEN SIAMA

Fairouz FEFEL

Mahdi BEN SOUISSI

Amine Taha Ben TITA

Ranim MAAREF

Mohamed Aziz MABROUKI

Supervisor:

Chaima Kaddour

Academic Year: 2024 - 2025

Contents

List of figures	3
1 Introduction	5
1.1 General Introduction	5
2 General Context of the Project	6
2.1 Introduction	6
2.2 Company Presentation	6
2.3 Overview	7
2.4 Core Values	7
2.5 Sectors of Activity	7
2.6 Technologies and Services	8
2.7 Website	8
2.8 Problem Statement	8
2.9 Existing Solutions	8
2.9.1 Evaluation of the CodingFactory Website	8
2.9.2 Comparison of Technical Performance	9
2.10 Proposed Solution	10
3 Requirements	11
3.1 Functional Requirements	11
3.1.1 User Management	11
3.1.2 Freelance Management	11
3.1.3 Project Management	12
3.1.4 Event Management	12
3.1.5 Final Year Project Management	12
3.1.6 Internship Management	13
3.1.7 Course Management	13
3.1.8 Complaint Management	13
3.1.9 Finance and Scholarship Management Module	14
3.2 Non-Functional Requirements	15

3.2.1	Performance	15
3.2.2	Security	15
3.2.3	Usability	15
3.2.4	Compatibility	15
3.3	Tools	16
3.3.1	Development Software	16
3.3.2	Version Control Systems	17
3.3.3	Project Management	18
3.3.4	Conception Software	18
4	Use Case Diagram	19
5	Class Diagram	20
6	Global Architecture of the Application	21
6.1	Overview	21
6.2	System Structure	21
6.3	Benefits of the Architecture	22
7	Mockups of the CodingFactory Platform	24
7.1	Homepage	24
7.2	Other Mockups	25
7.3	Login Page	26
7.4	Complaint Submission Page	27
8	Conclusion	30

List of Figures

2.1	Logo CodingFactory	6
2.2	Evaluation of the CodingFactory	9
2.3	Technical Performance	10
3.1	IntelliJ IDEA Logo	16
3.2	Apache HTTP Server Logo	16
3.3	Maven Logo	17
3.4	Logo MySQL	17
3.5	Logo GitHub	17
3.6	Logo Jira	18
3.7	Logo UML	18
4.1	Use Case Diagram	19
5.1	Class Diagram	20
6.1	Global Architecture of the Application	23
7.1	Mockup of the Homepage	25
7.2	Additional Mockup 1	26
7.3	Additional Mockup 2	27
7.4	Additional Mockup 3	28
7.5	Mockup of the Login Page	28
7.6	Mockup of the Complaint Submission Page	29

List of Tables

Introduction

1.1 General Introduction

In today's fast-paced digital world, managing various aspects of an organization efficiently requires robust and scalable solutions. The **CodingFactory Platform** aims to address these needs by providing an integrated system for managing users, freelancers, projects, events, final year projects, internships, courses, complaints, and financial resources.

This report documents the initial phase (**Sprint 0**) of the project, detailing the general context, problem statement, proposed solution, functional and non-functional requirements, system architecture, and preliminary design elements. By leveraging modern technologies and best software development practices, this platform aims to streamline operations and improve overall user experience.

General Context of the Project

2.1 Introduction

Before starting any project, a preliminary study is essential, as it facilitates the initiation of the work and ensures better organization. In this chapter, we will first situate our project within its general framework by presenting the host organization, its objectives, and its planning.

2.2 Company Presentation



Figure 2.1: Logo CodingFactory

2.3 Overview

Founded by the ESIEE-IT group, **CodingFactory** is a specialized institution dedicated to training students and professionals in software development and digital technologies. It provides hands-on, project-based learning experiences in programming, software engineering, and emerging technologies to meet the growing demands of the job market.

CodingFactory focuses on an immersive pedagogy, equipping learners with industry-relevant skills and preparing them for careers in software development, web technologies, and digital transformation.

2.4 Core Values

- **Excellence** – Providing high-quality training and cutting-edge expertise
- **Innovation** – Encouraging creativity and the adoption of new technologies
- **Collaboration** – Fostering teamwork and collective intelligence
- **Agility** – Adapting quickly to industry trends and technological evolution
- **Professionalism** – Preparing students for real-world challenges

2.5 Sectors of Activity

CodingFactory specializes in IT training and education, offering expertise in various digital fields, including:

- Software Development
- Web and Mobile Applications
- Cloud Computing
- Artificial Intelligence
- Cybersecurity
- DevOps and Agile Methodologies

2.6 Technologies and Services

The institution offers training and professional certifications in several key technologies:

- **Programming Languages:** Java, Python, JavaScript, TypeScript, C#
- **Web and Frontend Development:** Angular, React, Vue.js
- **Backend Development:** Spring Boot, Node.js, .NET Core
- **Databases and Big Data:** MySQL, PostgreSQL, MongoDB, Hadoop
- **DevOps and Cloud:** Docker, Kubernetes, AWS, Azure

These training programs aim to prepare students for dynamic and high-demand careers in technology and digital services.

2.7 Website

For more details, visit: <https://codingfactory.fr>

2.8 Problem Statement

How can we modernize and optimize the *CodingFactory* website to enhance user experience, accessibility, and automation while efficiently managing academic resources, student projects, internships, events, and professional opportunities? Additionally, how can we ensure seamless role-based access, real-time collaboration, and integration of essential features while addressing challenges such as outdated interfaces, fragmented systems, and limited scalability?

2.9 Existing Solutions

2.9.1 Evaluation of the CodingFactory Website

This mind map provides an evaluation of the CodingFactory website, analyzing key aspects such as program information, student life & events, transparency, student feedback, and navigation & usability. It highlights strengths like detailed program descriptions and

good response time but also points out areas for improvement, such as lack of student engagement, missing admission criteria, absence of a code archiving system, and the need for better student feedback integration. The study suggests enhancements to create a more engaging and transparent user experience.

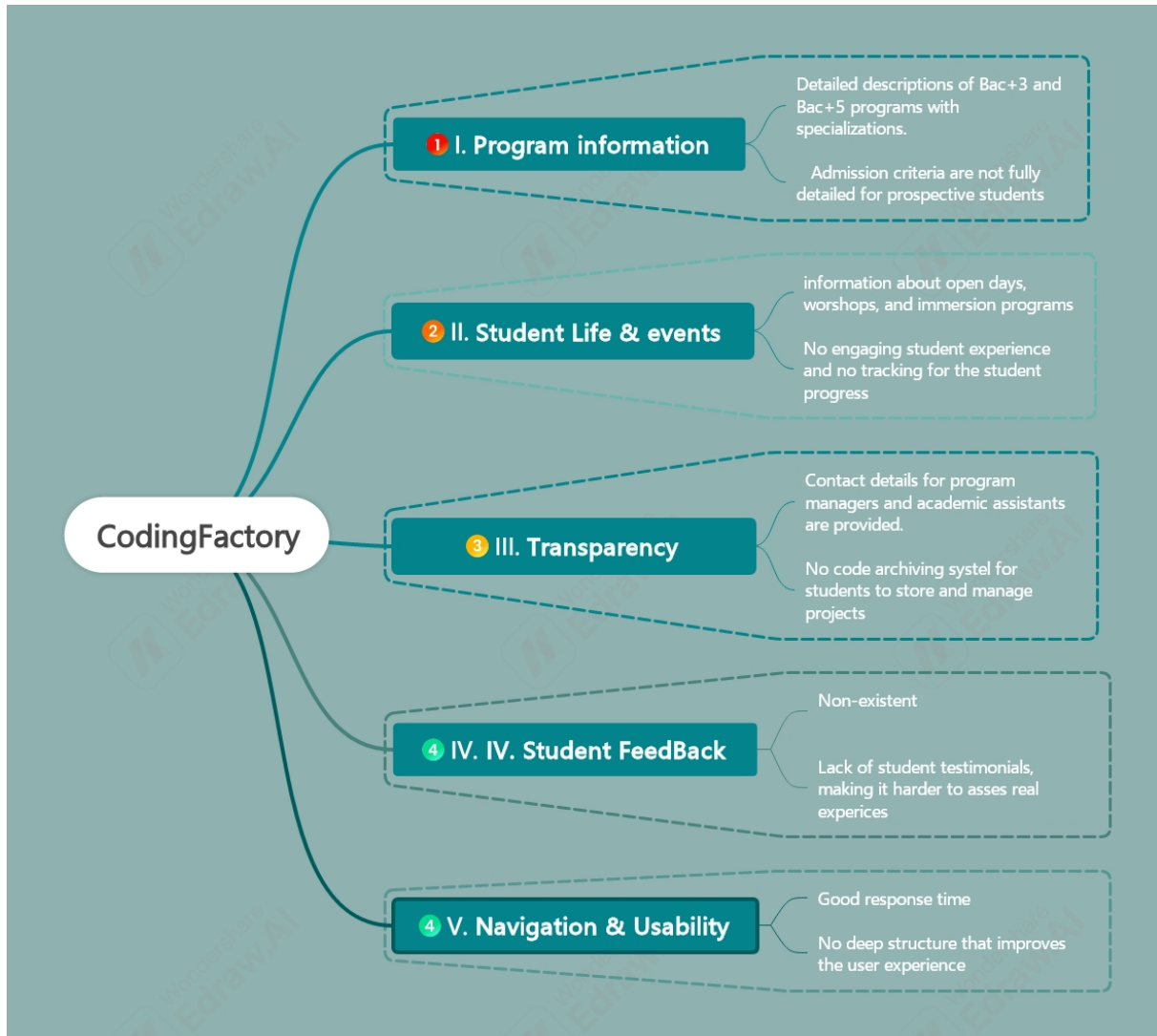


Figure 2.2: Evaluation of the CodingFactory

2.9.2 Comparison of Technical Performance

This column chart compares the technical performance of various coding school websites, evaluating factors such as average response time, page size, number of HTTP requests, and overall performance scores. The websites analyzed include 42, Web@cademie, General Assembly, Simplon.co, and Wild Code School, with varying scores ranging from 7.25 to 14. The visualization highlights differences in optimization and efficiency, offering insights

into which platforms perform better in terms of speed and responsiveness.

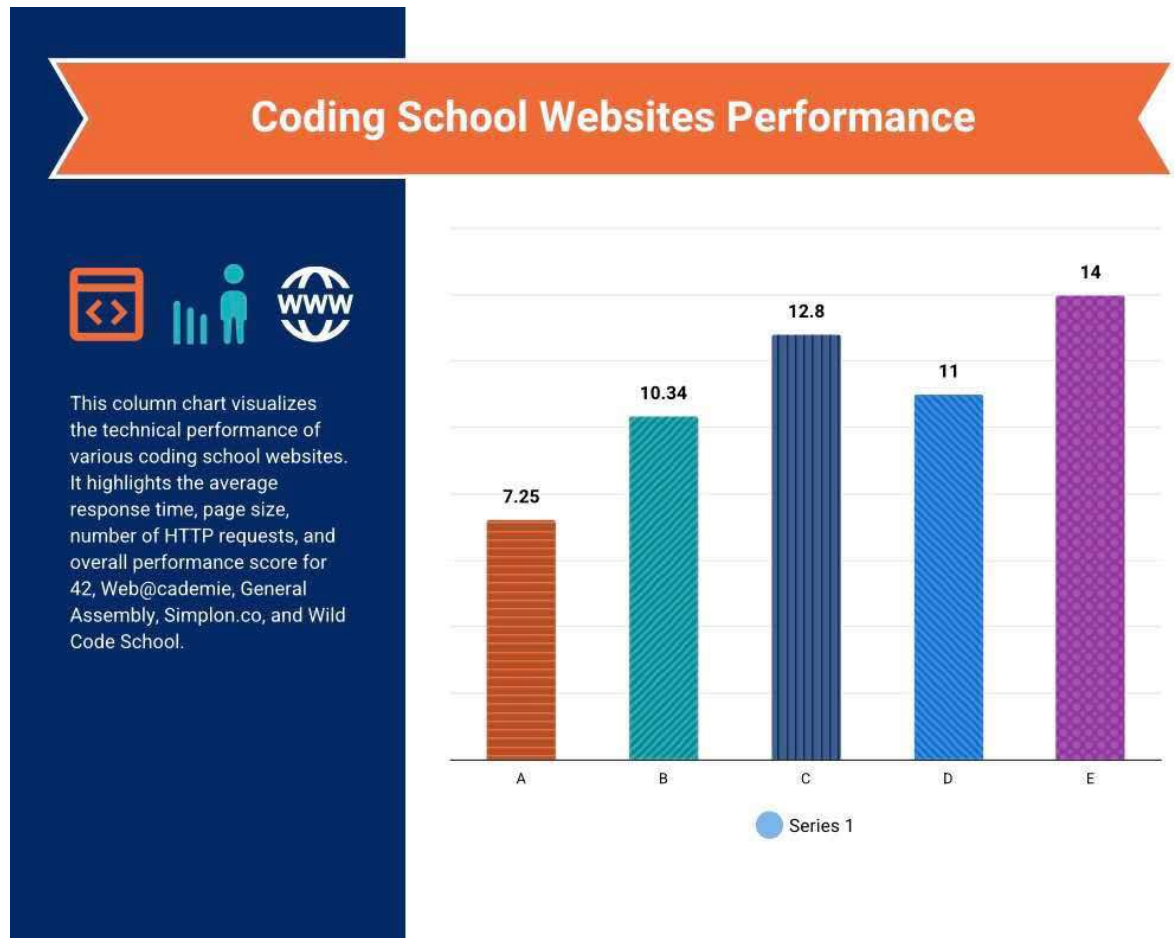


Figure 2.3: Technical Performance

2.10 Proposed Solution

To address these challenges, our project introduces a **revamped and modernized CodingFactory web platform** designed to **streamline user interactions, automate essential academic processes, and enhance collaboration** among students and tutors. The new platform will provide a **centralized and intuitive system** with **role-based authentication, improved accessibility, and modular features** for training management, internships, final-year projects, freelancing opportunities, and events. By integrating automation, scalability, and a user-friendly interface, this solution will improve efficiency, engagement, and digital transformation for the CodingFactory ecosystem.

Requirements

3.1 Functional Requirements

3.1.1 User Management

- **Registration:** Allow users to sign up with unique usernames and passwords.
- **Authentication:** Enable secure login using validated credentials.
- **Profile Editing:** Allow users to update personal information and upload profile pictures.
- **Role-Based Access:** Implement different roles (admin, instructor, student) with specific permissions.
- **Account Management:** Enable admins to manage user accounts, including activation and deactivation.
- **Security Measures:** Implement CAPTCHA, encryption, and strong password policies.

3.1.2 Freelance Management

- **Job Posting:** Allow companies or clients to post freelance job offers.
- **Application Management:** Enable students to apply for freelance opportunities.
- **Profile Showcase:** Let students create and showcase their freelance portfolios.

- Payment Integration: Secure payment handling between freelancers and clients.
- Rating & Reviews: Implement a review system for freelancers and clients.

3.1.3 Project Management

- Project Creation: Allow users to create and manage projects with milestones.
- Task Assignment: Enable team collaboration and task delegation.
- Collaboration Tools: Integrate messaging, file sharing, and real-time updates.
- Progress Tracking: Monitor task completion and deadlines.
- Project Documentation: Store and manage project-related files securely.
- Project Recommendation: Recommend projects according to the student profile.

3.1.4 Event Management

- Event Creation: Allow admins to create hackathons, workshops, and meetups.
- Registration System: Enable students to register and track event participation.
- Scheduling & Notifications: Send reminders and updates about events.
- Feedback System: Collect post-event reviews and suggestions.

3.1.5 Final Year Project Management

- Proposal Submission: Allow students to submit project proposals.
- Project Approval & Feedback: Enable instructors to review and approve projects.
- Supervision & Progress Tracking: Assign mentors and track student progress.
- Milestone Management: Set deadlines and evaluate different project phases.
- Plagiarism Check: Integrate tools to ensure originality of work.
- Final Evaluation & Grading: Implement a structured evaluation process.

3.1.6 Internship Management

- **Internship Listings:** Provide a database of available internships.
- **Application & Matching:** Allow students to apply and match with suitable internships.
- **Internship Tracking:** Enable supervisors to monitor interns' progress.
- **Reporting & Certification:** Allow students to submit reports and receive completion certificates.

3.1.7 Course Management

- **Course Creation:** Allow admins/instructors to create courses with descriptions and objectives.
- **Content Management:** Allow instructors to add, edit, and organize course materials.
- **Progress Tracking:** Provide tools for monitoring course progress and grades.
- **Administration:** Grant admins oversight of course activities and user permissions.
- **Accessibility:** Ensure the system is accessible and compatible across devices.

3.1.8 Complaint Management

- **Complaint Submission:** Allow students, instructors, and staff to submit complaints.
- **Category Selection:** Enable users to classify complaints (academic, administrative, technical issues, etc.).
- **Status Tracking:** Allow users to track the resolution progress of their complaints.
- **Priority Management:** Classify complaints based on urgency and impact.
- **Automated Assignment:** Route complaints to the appropriate department or personnel.
- **Resolution Management:** Enable staff to provide updates and resolutions.

- **Anonymous Complaints:** Allow users to submit anonymous reports for sensitive issues.
- **Escalation System:** Automatically escalate unresolved complaints to higher authorities.
- **Feedback and Satisfaction Rating:** Let users provide feedback on complaint handling.
- **Reporting and Analytics:** Generate reports to analyze trends and improve service quality.

3.1.9 Finance and Scholarship Management Module

- **Fee Management:** Handle tuition payments, due dates, and invoices.
- **Online Payment Integration:** Allow secure transactions via credit card, PayPal, or bank transfers.
- **Scholarship Application:** Enable students to apply for scholarships with document uploads.
- **Eligibility Verification:** Implement automatic eligibility checks based on predefined criteria.
- **Grant Allocation:** Manage the distribution of scholarships and financial aid.
- **Installment Plans:** Allow students to pay tuition in flexible installments.
- **Financial Reports:** Generate financial statements, balance sheets, and expense tracking.
- **Tax and Deductions:** Handle tax reporting for tuition fees and scholarships.
- **Reimbursement System:** Process refund requests and manage approvals.
- **Audit and Compliance:** Ensure transparency in financial operations with logs and audits.

3.2 Non-Functional Requirements

3.2.1 Performance

- The system should be responsive and load quickly, even under heavy traffic loads.
- Implement caching mechanisms for frequently accessed data to reduce response times by identifying frequently accessed data, choosing a caching provider, configuring caching settings, annotating methods for caching, tuning cache parameters, and thoroughly testing and monitoring performance.

3.2.2 Security

- Data transmission should be encrypted using HTTPS to ensure secure communication between the client and server.
- User authentication and authorization should be implemented securely, using Spring Security for user authentication and authorization, ensuring password hashing and salting for secure storage.
- Protection against common web vulnerabilities such as SQL injection should be incorporated. Use frameworks like Hibernate or JPA for ORM to prevent SQL injection attacks.

3.2.3 Usability

- The user interface should be intuitive and easy to navigate, with consistent design patterns and clear navigation menus.
- Design a clean and intuitive user interface using Angular Material or other UI libraries for Angular.

3.2.4 Compatibility

- The application should be compatible with a wide range of devices and browsers, including desktops, laptops, tablets, and mobile phones.
- Stay updated with modern web standards and technologies, such as HTML5 and CSS3, to maintain compatibility across platforms.

3.3 Tools

3.3.1 Development Software

- **Integrated Development Environment (IDE):** IntelliJ IDEA, a powerful integrated development environment used for developing applications in Java and other languages.

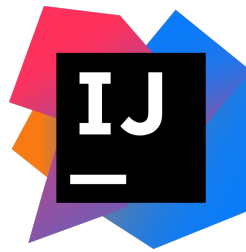


Figure 3.1: IntelliJ IDEA Logo

- **Web Servers:** Apache HTTP Server, an open-source web server widely used for hosting applications and serving web content.



Figure 3.2: Apache HTTP Server Logo

- **Package Managers:** Maven, a build automation and dependency management tool primarily used for Java projects.



Figure 3.3: Maven Logo

- **MySQL :** Système de gestion de base de données relationnelle utilisé pour stocker les informations relatives aux produits, mouvements de stock et utilisateurs.



Figure 3.4: Logo MySQL

3.3.2 Version Control Systems

- **Git & GitHub:** A distributed version control system that allows developers to track changes, collaborate on projects, and manage code efficiently.



Figure 3.5: Logo GitHub

3.3.3 Project Management

- Jira: A project management tool that helps teams track, manage, and prioritize tasks and projects in an agile environment.



Figure 3.6: Logo Jira

3.3.4 Conception Software

UML (*Unified Modeling Language*) is a standardized visual language used to model software systems, representing structure (class diagrams), behavior (use case, sequence, activity diagrams), and interactions within a system.



Figure 3.7: Logo UML

Use Case Diagram

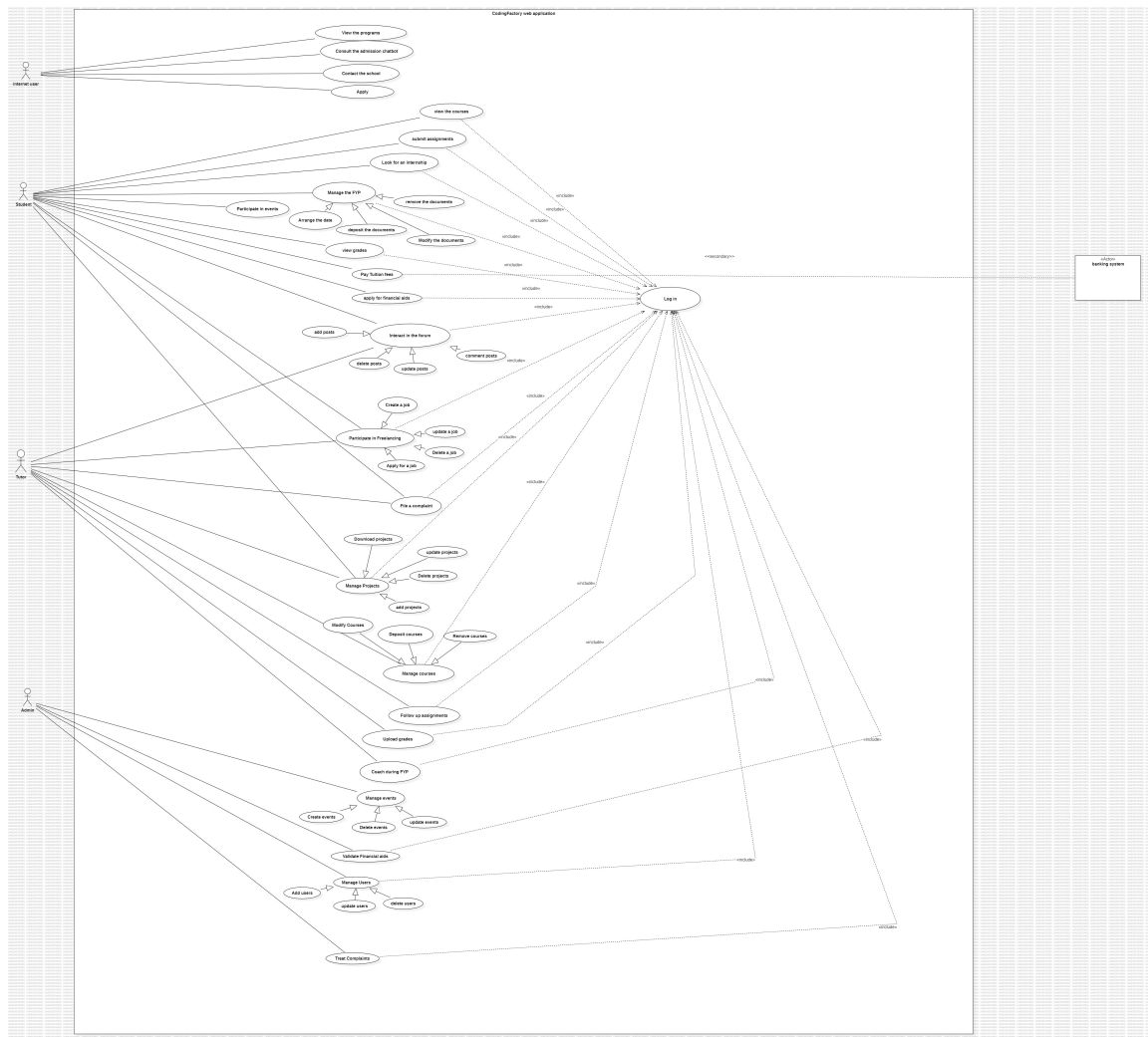


Figure 4.1: Use Case Diagram

Class Diagram

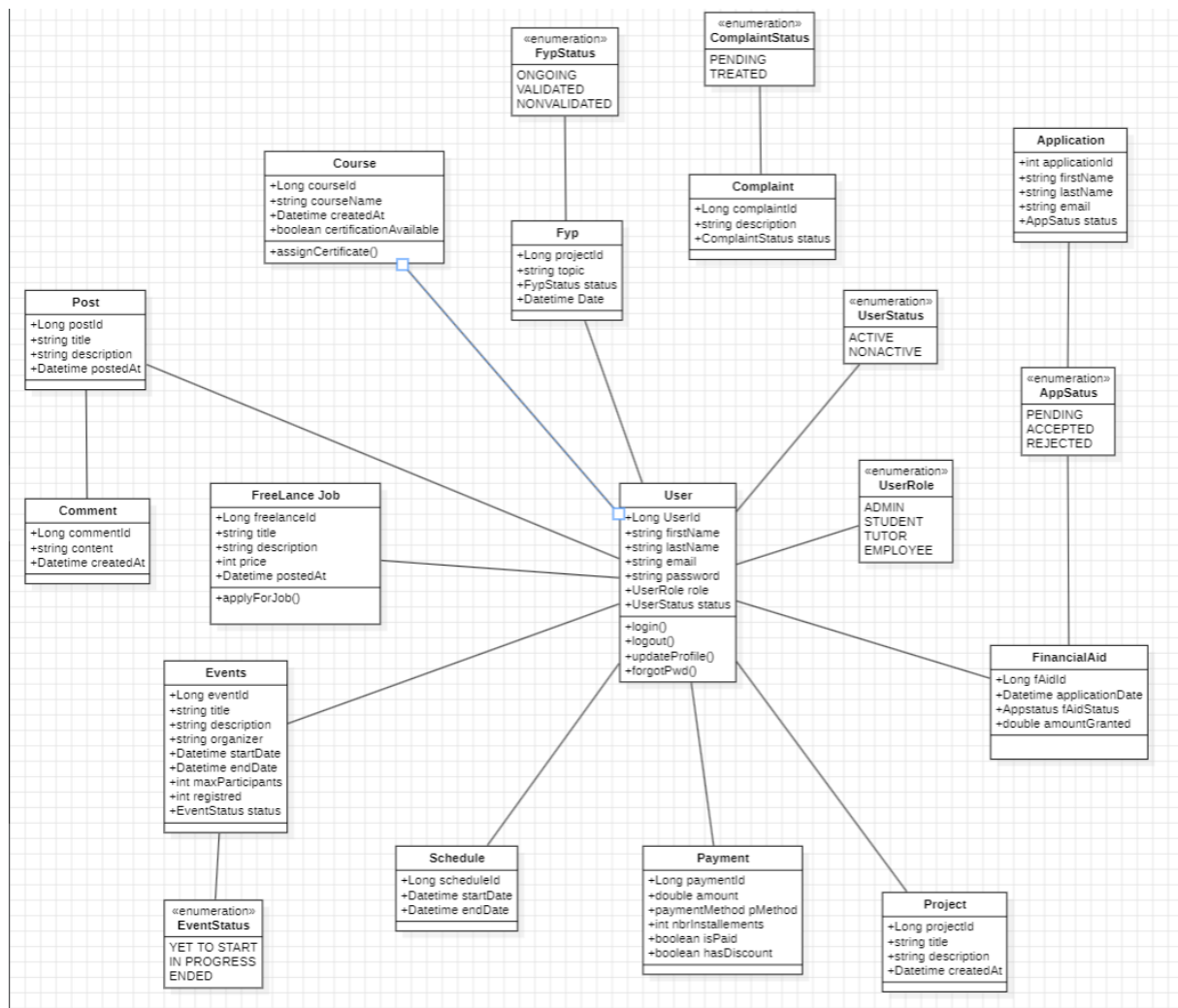


Figure 5.1: Class Diagram

Global Architecture of the Application

6.1 Overview

Our web application follows a **microservices architecture** using **Spring Boot** with **JPA** for backend services and **MySQL** for data storage. This approach ensures scalability, flexibility, and maintainability by allowing services to operate independently while being efficiently managed through an API Gateway.

6.2 System Structure

The system is structured as follows:

- **Frontend Client:** The user interacts with the application through a web-based frontend.
- **API Gateway & Load Balancer:** Acts as an entry point, managing incoming requests and distributing traffic across various microservices.
- **Microservices Layer:** Each core functionality is handled by an independent **Spring Boot** microservice:
 - Course Management
 - Events
 - Resources
 - User Management

- Internships
 - Forum
 - Complaints
 - Finance
- **Database Layer:** Each microservice has its dedicated **MySQL** database for data persistence, ensuring modularity and scalability.

6.3 Benefits of the Architecture

This microservices architecture provides the following advantages:

- **Scalability:** Each service can be independently scaled based on demand.
- **Flexibility:** Services can be updated or replaced without affecting the entire system.
- **Maintainability:** Code is modular and easier to manage, reducing complexity.

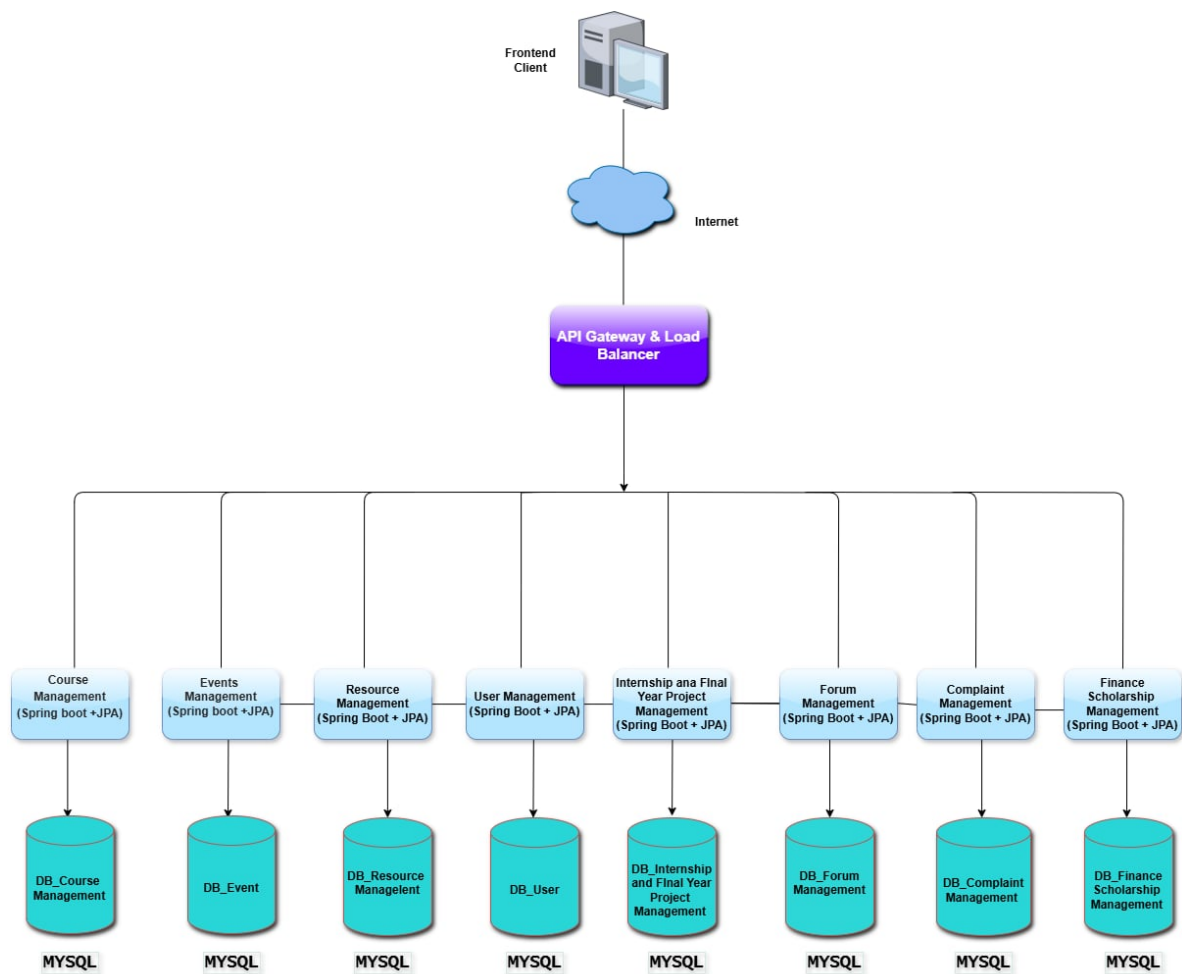


Figure 6.1: Global Architecture of the Application

Mockups of the CodingFactory Platform

This chapter presents the graphical user interface (GUI) mockups for the CodingFactory platform, showcasing key pages such as the homepage, login, and complaint submission.

7.1 Homepage

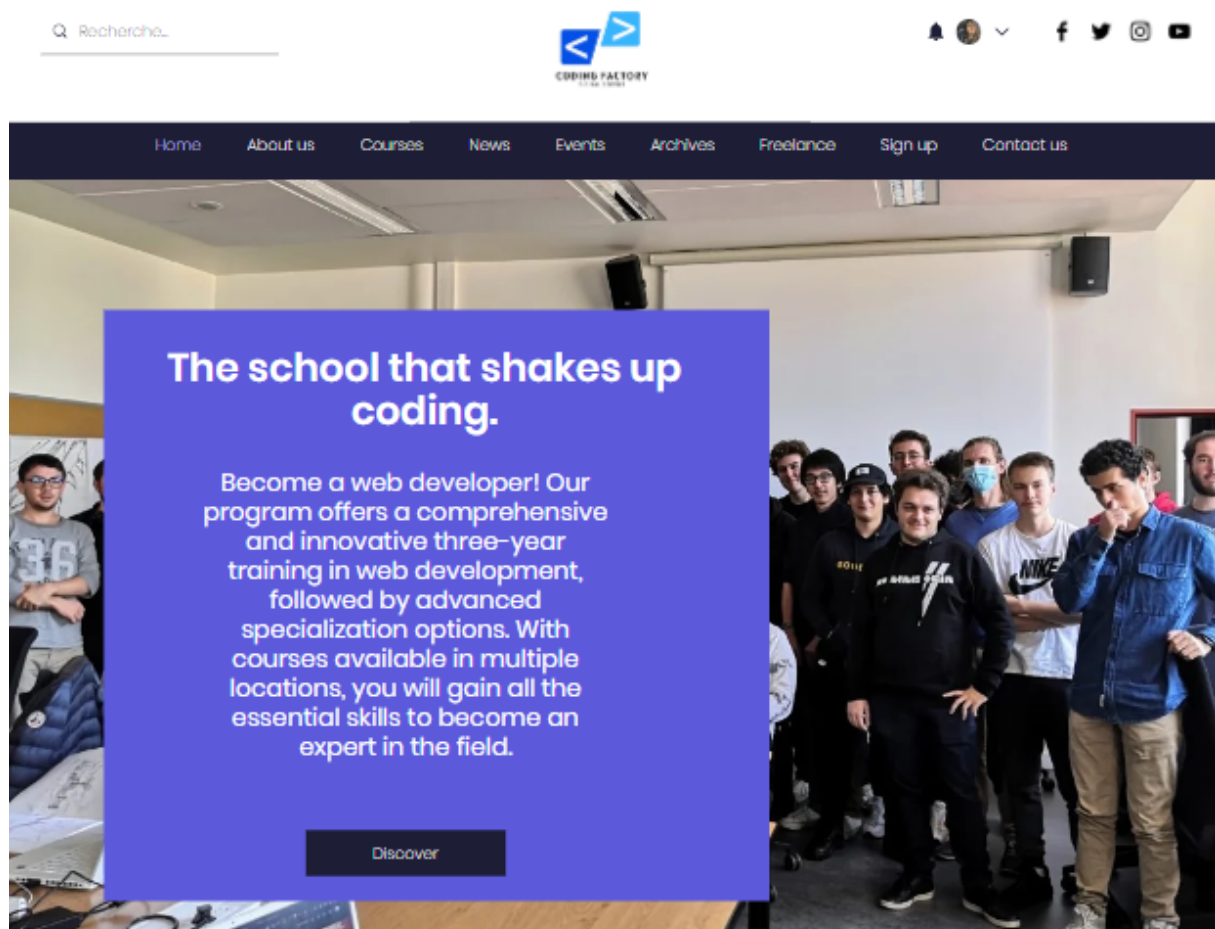


Figure 7.1: Mockup of the Homepage

7.2 Other Mockups

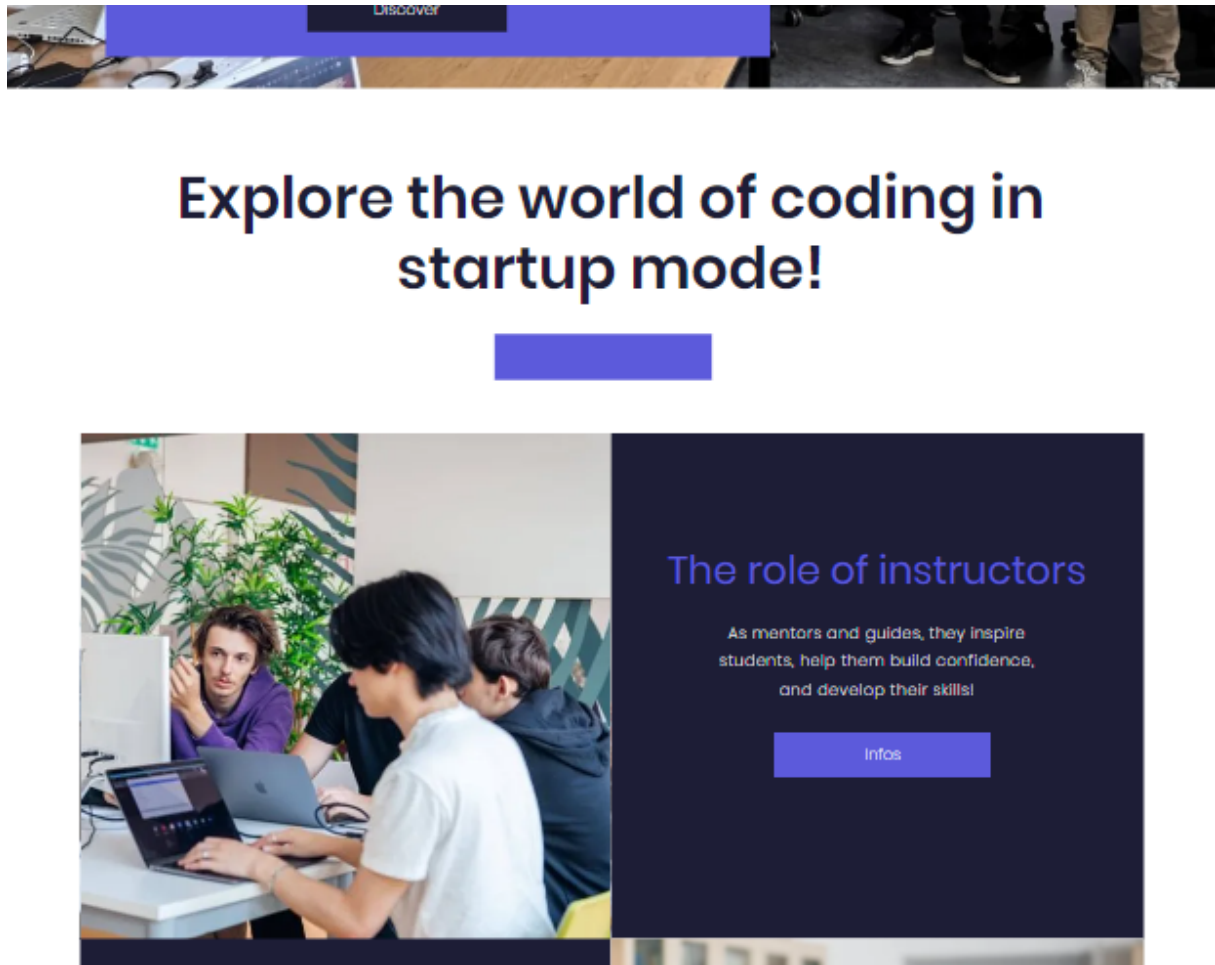


Figure 7.2: Additional Mockup 1

7.3 Login Page

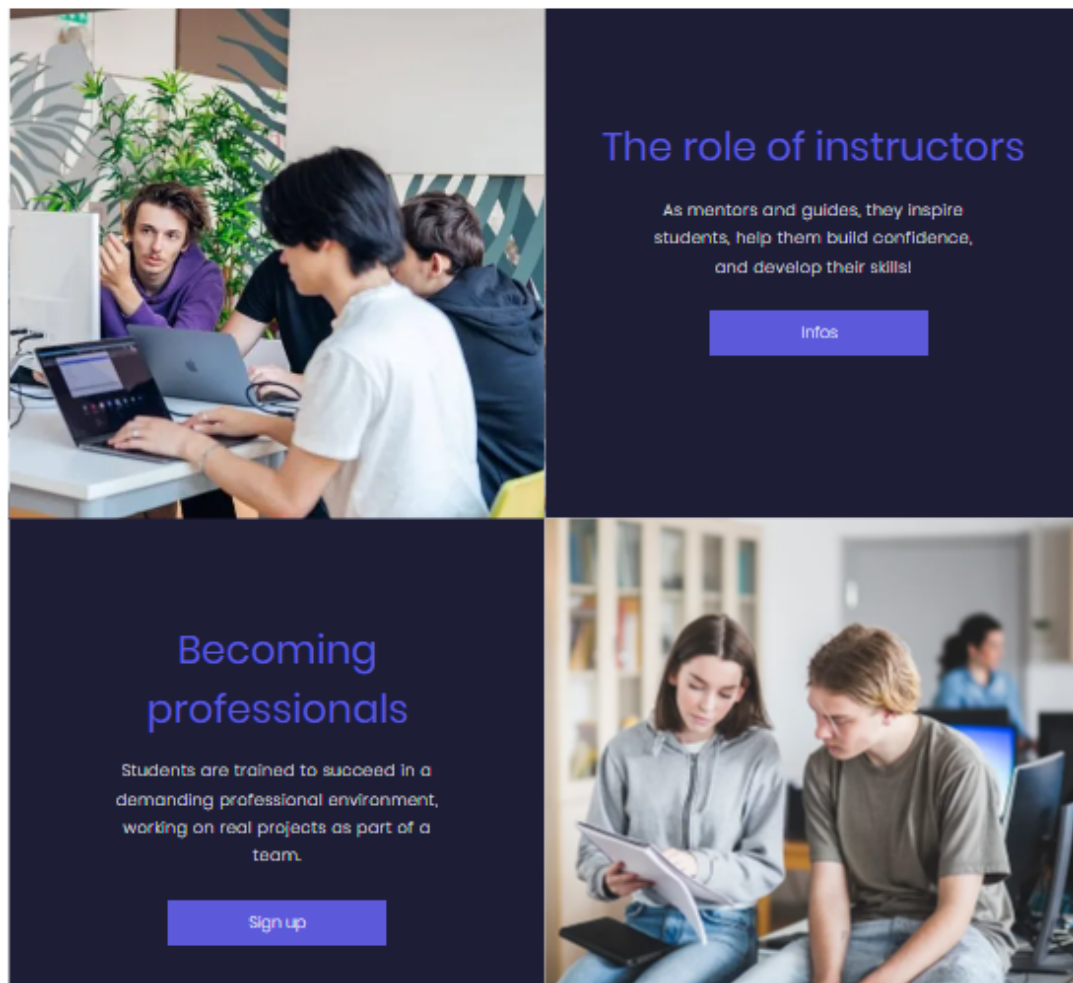


Figure 7.3: Additional Mockup 2

7.4 Complaint Submission Page

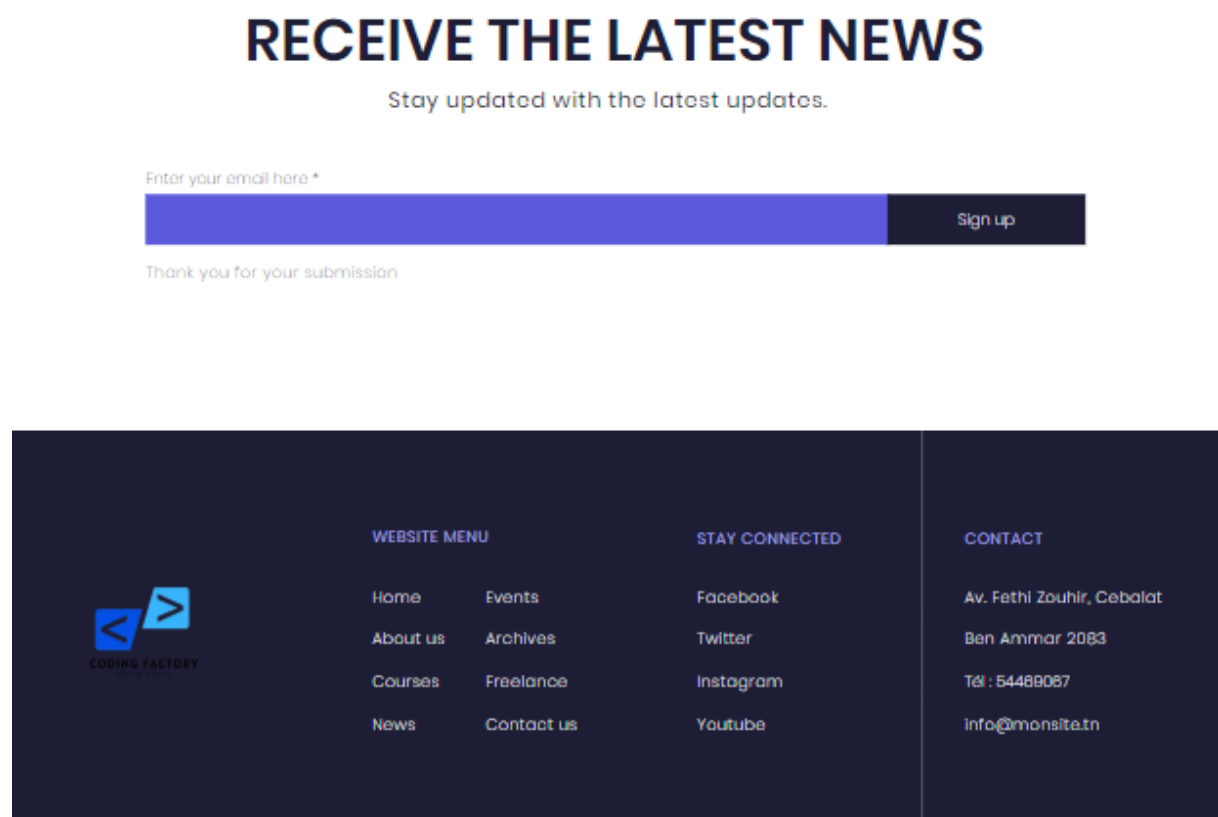


Figure 7.4: Additional Mockup 3

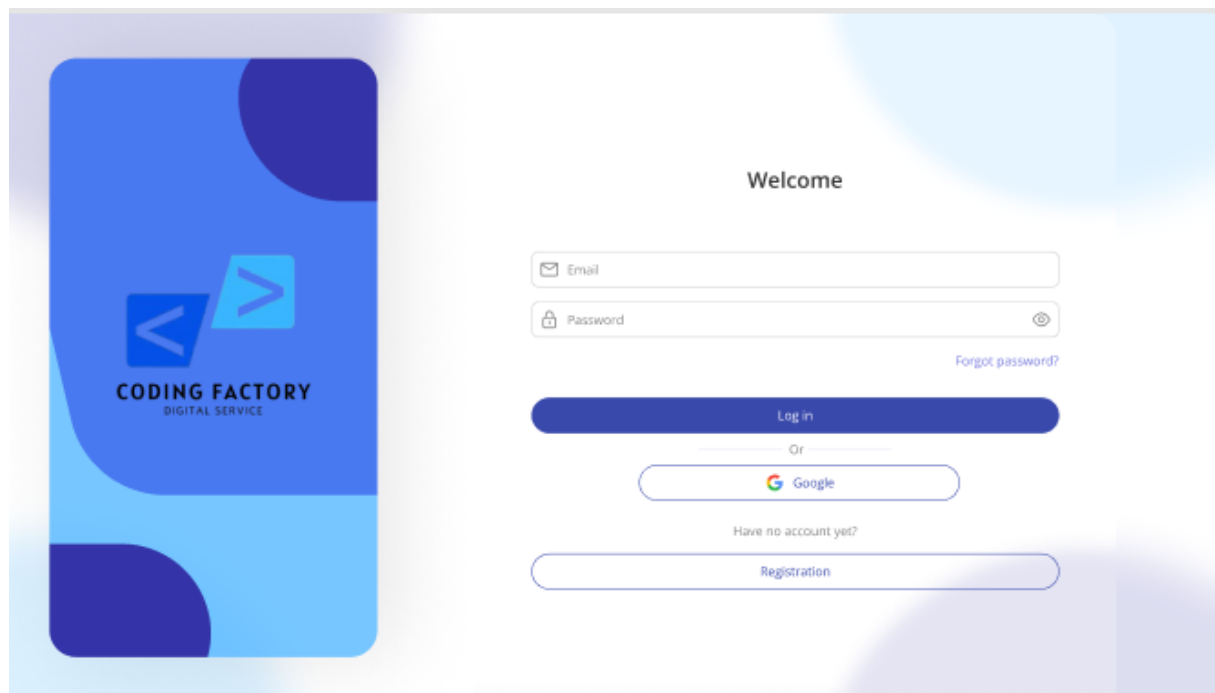
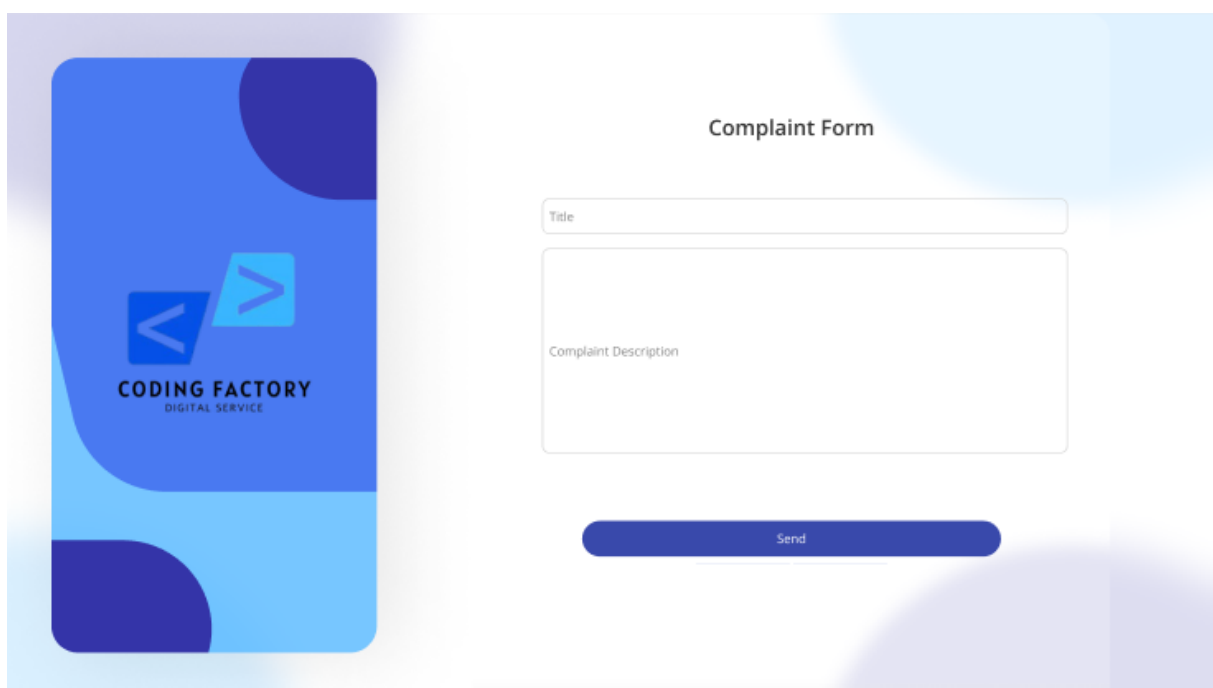


Figure 7.5: Mockup of the Login Page



The mockup displays a 'Complaint Form' on a light blue background. On the left is a vertical blue sidebar with the 'CODING FACTORY DIGITAL SERVICE' logo, which includes two stylized code symbols: a blue less-than sign (<) and a light blue greater-than sign (>). The form itself is white and contains a 'Title' input field, a larger 'Complaint Description' text area, and a dark blue 'Send' button at the bottom.

Complaint Form

Title

Complaint Description

Send

Figure 7.6: Mockup of the Complaint Submission Page

Conclusion

The **CodingFactory Platform** represents a significant step toward simplifying and optimizing project and resource management in an organizational setting. Through this sprint, we have established a clear vision for the system by defining its core requirements, architecture, and user interface design.

Moving forward, subsequent sprints will focus on implementing the functional components, refining the user experience, and ensuring the platform meets performance, security, and usability standards. By adopting an iterative development approach, we aim to deliver a high-quality solution that effectively addresses the identified challenges and provides long-term value to its users.